Implementation Note: Mass-Spring Simulation with Moving Dirichlet Boundary Condition

April 2026

1 Overview

We simulate a 2D elastic square falling onto a ground under gravity and then compressed by a moving ceiling. The solid is discretized as a mass-spring system on a regular grid. Each time step is solved by minimizing the energy using the Newton method with line search.

2 Degrees of Freedom

The system has $n = n_{\text{mesh}} + 1$ nodes, where $n_{\text{mesh}} = (n_{\text{seg}} + 1)^2$ are the mesh nodes of the square, and the additional node represents the ceiling. Each node i has position $\boldsymbol{x}_i \in \mathbb{R}^2$, giving a total of $2n$ degrees of freedom.

3 Incremental Potential

At each time step, we solve:

$$\boldsymbol{x}^{n+1} = \arg\min_{\boldsymbol{x}} \underbrace{\frac{1}{2h^2}\sum_i m_i \|\boldsymbol{x}_i - \tilde{\boldsymbol{x}}_i\|^2}_{\text{Inertia}} + \underbrace{E_{\text{grav}}(\boldsymbol{x}) + E_{\text{spring}}(\boldsymbol{x}) + E_{\text{barrier}}(\boldsymbol{x}) + E_{\text{friction}}(\boldsymbol{x})}_{\text{Potentials}} + \underbrace{E_{\text{DBC}}(\boldsymbol{x})}_{\text{Penalty}}$$

where $\tilde{\boldsymbol{x}}_i = \boldsymbol{x}_i^n + h\boldsymbol{v}_i^n$ is the predictive position from implicit Euler, and h is the time step size.

Note: in our Taichi implementation, the IP is scaled by $1/h^2$ relative to the standard formulation. All potentials keep their natural form, while the inertia term becomes $\frac{m_i}{2h^2}\|\boldsymbol{x}_i - \tilde{\boldsymbol{x}}_i\|^2$ and the DBC penalty also carries a $1/h^2$ factor.

3.1 Inertia Energy

$$E_{\text{inertia}} = \sum_i \frac{m_i}{2h^2}\|\boldsymbol{x}_i - \tilde{\boldsymbol{x}}_i\|^2$$

Gradient: $\nabla_{\boldsymbol{x}_i} E_{\text{inertia}} = \frac{m_i}{h^2}(\boldsymbol{x}_i - \tilde{\boldsymbol{x}}_i)$
Hessian: $\nabla^2_{\boldsymbol{x}_i} E_{\text{inertia}} = \frac{m_i}{h^2}\boldsymbol{I}_{2\times 2}$

3.2 Gravity Energy

$$E_{\text{grav}} = -\sum_i m_i\, \boldsymbol{x}_i \cdot \boldsymbol{g}, \quad \boldsymbol{g} = (0, -9.8)$$

Gradient: $\nabla_{\boldsymbol{x}_i} E_{\text{grav}} = -m_i \boldsymbol{g}$
Hessian: $\boldsymbol{0}$

3.3 Mass-Spring Energy

For each edge $e = (i, j)$ with rest length squared l_e^2 and stiffness k_e:

$$E_{\text{ms}} = \sum_e \frac{1}{2} k_e l_e^2 \left(\frac{\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2}{l_e^2} - 1 \right)^2$$

Let $\boldsymbol{d} = \boldsymbol{x}_i - \boldsymbol{x}_j$ and $s = \|\boldsymbol{d}\|^2 / l_e^2 - 1$.
Gradient:

$$\nabla_{\boldsymbol{x}_i} E_e = 2k_e s\, \boldsymbol{d}, \quad \nabla_{\boldsymbol{x}_j} E_e = -2k_e s\, \boldsymbol{d}$$

The 2×2 edge Hessian block (before PSD projection):

$$\boldsymbol{H_d} = \frac{2k_e}{l_e^2} \left(2\boldsymbol{d}\boldsymbol{d}^T + (\|\boldsymbol{d}\|^2 - l_e^2)\boldsymbol{I} \right)$$

The 4×4 assembled Hessian for the edge is:

$$\begin{pmatrix} \boldsymbol{H_d} & -\boldsymbol{H_d} \\ -\boldsymbol{H_d} & \boldsymbol{H_d} \end{pmatrix}$$

PSD Projection. When springs are compressed ($\|\boldsymbol{d}\|^2 < l_e^2$), the Hessian may have negative eigenvalues. We project each 2×2 block to be positive semi-definite by computing its eigendecomposition analytically:

$$\lambda_{0,1} = \frac{\operatorname{tr}(\boldsymbol{H_d})}{2} \pm \sqrt{\frac{\operatorname{tr}(\boldsymbol{H_d})^2}{4} - \det(\boldsymbol{H_d})}$$

and clamping $\lambda_i \leftarrow \max(\lambda_i, 0)$, then reconstructing $\boldsymbol{H_d} = \boldsymbol{V} \operatorname{diag}(\lambda_0, \lambda_1) \boldsymbol{V}^T$.

3.4 Barrier Energy (Ground)

For each node i with distance to ground $d_i = \boldsymbol{n} \cdot (\boldsymbol{x}_i - \boldsymbol{o})$, where $\boldsymbol{n}$ is the ground normal and $\boldsymbol{o}$ is a point on the ground:

$$E_{\text{barrier}} = \sum_{i:\, 0 < d_i < \hat{d}} \frac{w_i \hat{d} \kappa}{2} (s_i - 1) \ln(s_i), \quad s_i = \frac{d_i}{\hat{d}}$$

where w_i is the contact area and $\hat{d}$ is the activation distance.
Gradient (letting b' denote $\partial E / \partial d$):

$$b'(d) = \frac{w \hat{d} \kappa}{2} \left(\frac{\ln s}{\hat{d}} + \frac{s - 1}{d} \right), \quad \nabla_{\boldsymbol{x}_i} E_{\text{barrier}} = b'(d_i)\, \boldsymbol{n}$$

Hessian:

$$b''(d) = \frac{w\kappa}{2} \cdot \frac{d + \hat{d}}{d^2}, \quad \nabla^2_{\boldsymbol{x}_i} E_{\text{barrier}} = b''(d_i)\, \boldsymbol{n}\boldsymbol{n}^T$$

(The second-order term $b'(d) \nabla^2 d$ vanishes because d is linear in $\boldsymbol{x}$.)

3.5 Barrier Energy (Ceiling)

The ceiling is a moving node with index c and downward normal $\boldsymbol{n}_c = (0, -1)$. For each mesh node i, the distance is $d_i = \boldsymbol{n}_c \cdot (\boldsymbol{x}_i - \boldsymbol{x}_c)$.

The energy, gradient, and Hessian formulas are identical to the ground barrier, but because the ceiling is also a degree of freedom, the gradient has a *reaction term* on the ceiling node:

$$\nabla_{\boldsymbol{x}_i} E = b'(d_i)\, \boldsymbol{n}_c, \quad \nabla_{\boldsymbol{x}_c} E = -b'(d_i)\, \boldsymbol{n}_c$$

The Hessian is a 4×4 block involving nodes i and c:

$$\begin{pmatrix} +b''\boldsymbol{n}_c\boldsymbol{n}_c^T & -b''\boldsymbol{n}_c\boldsymbol{n}_c^T \\ -b''\boldsymbol{n}_c\boldsymbol{n}_c^T & +b''\boldsymbol{n}_c\boldsymbol{n}_c^T \end{pmatrix}$$

3.6 Friction Energy

For each node in contact with the ground ($\mu\lambda_i > 0$), we compute the tangential velocity $\bar{\boldsymbol{v}}_i = \boldsymbol{T}\boldsymbol{v}_i$ where $\boldsymbol{T} = \boldsymbol{I} - \boldsymbol{n}\boldsymbol{n}^T$ and $\boldsymbol{v}_i = (\boldsymbol{x}_i - \boldsymbol{x}_i^n)/h$:

$$E_{\text{friction}} = \sum_i \mu\lambda_i\, f_0(\|\bar{\boldsymbol{v}}_i\|, h)$$

The function f_0 is a smoothed approximation (with parameter ϵ_v) to avoid the non-differentiability at zero sliding velocity:

$$f_0(y, h) = \begin{cases} yh & \text{if } yh \geq \epsilon_v h \\ -\frac{(yh)^3}{3(\epsilon_v h)^2} + \frac{(yh)^2}{\epsilon_v h} + \frac{\epsilon_v h}{3} & \text{otherwise} \end{cases}$$

The friction coefficient $\mu\lambda_i$ is computed at the beginning of each time step using the barrier gradient at $\boldsymbol{x}^n$ (lagged friction).

A more detailed explanation can be viewed at the folder for friction.

3.7 DBC Penalty (Spring Energy)

The ceiling is driven to a target position $\boldsymbol{x}_c^*$ via a penalty spring:

$$E_{\text{DBC}} = \frac{k_{\text{DBC}}\, m_c}{2h^2} \|\boldsymbol{x}_c - \boldsymbol{x}_c^*\|^2$$

The target is computed each time step as:

$$\boldsymbol{x}_c^* = \begin{cases} \boldsymbol{x}_c^n + h\, \boldsymbol{v}_{\text{DBC}} & \text{if } (\boldsymbol{x}_{\text{limit}} - \boldsymbol{x}_c^n) \cdot \boldsymbol{v}_{\text{DBC}} > 0 \\ \boldsymbol{x}_c^n & \text{otherwise (limit reached)} \end{cases}$$

The stiffness k_{DBC} is *warm-started* across time steps and doubled whenever the Newton solver converges but the ceiling has not reached its target.

4 Newton Solver

Each time step is solved by the projected Newton method:

Algorithm 1 Newton Step for One Time Step

```
x ← x^n, compute x̃ = x^n + h v^n, compute μλ from x^n
Compute ceiling target x_c^*
for iter = 1, ..., N_max do
    Check DBC: if ‖x_c − x_c^*‖/h < tol, freeze ceiling DOF
    Compute E(x) and ∇E(x)
    r ← max_i |∇_{x_i} E|/h over free nodes
    if r < tol and DBC satisfied then
        break (converged)
    else if r < tol and DBC not satisfied then
        k_DBC ← 2 k_DBC, continue
    end if
    Assemble Hessian H (with PSD projection and DOF elimination)
    Solve Hp = −∇E
    α ← CCD filter (floor + ceiling)
    Line search: find α such that E(x + αp) ≤ E(x) + c α ∇E · p
    x ← x + αp
end for
v^{n+1} = (x^{n+1} − x^n)/h
```

4.1 DOF Elimination

For Dirichlet nodes (frozen ceiling), we set the Hessian row/column to identity and the gradient to zero. This is done during assembly: for each energy term, if node i has `is_dbc`$[i] = 1$, we skip its off-diagonal Hessian entries and gradient contributions.

4.2 Continuous Collision Detection (CCD)

Before line search, we compute a maximum safe step size α to prevent any node from penetrating the ground or ceiling in one step.

For the ground (fixed):

$$\alpha \leq \frac{0.9\, d_i}{-\boldsymbol{p}_i \cdot \boldsymbol{n}} \quad \text{for all } i \text{ with } \boldsymbol{p}_i \cdot \boldsymbol{n} < 0$$

For the ceiling (moving), we use relative displacement:

$$\alpha \leq \frac{0.9\, d_i}{-(\boldsymbol{p}_i - \boldsymbol{p}_c) \cdot \boldsymbol{n}_c} \quad \text{for all } i \text{ with } (\boldsymbol{p}_i - \boldsymbol{p}_c) \cdot \boldsymbol{n}_c < 0$$

The factor 0.9 ensures nodes do not land exactly on the surface (where the barrier's $\ln(0)$ would diverge).

4.3 Backtracking Line Search

After computing the Newton direction $\boldsymbol{p}$ by solving

$$\mathbf{H}\boldsymbol{p} = -\nabla E, \tag{1}$$

we determine a suitable step size $\alpha \in (0, 1]$ using backtracking line search.

We adopt the Armijo condition to ensure sufficient decrease in the energy:

$$E(\boldsymbol{x} + \alpha \boldsymbol{p}) \leq E(\boldsymbol{x}) + c\alpha \nabla E(\boldsymbol{x})^T \boldsymbol{p}, \tag{2}$$

where $c \in (0, 1)$ is a small constant (we use $c = 10^{-4}$).

The term $\nabla E(\boldsymbol{x})^T \boldsymbol{p}$ represents the first-order directional derivative, i.e., the predicted decrease of the energy along direction $\boldsymbol{p}$.

Before backtracking, we also apply Continuous Collision Detection (CCD) to cap the step size:

$$\alpha \leftarrow \min(\alpha, \alpha_{\text{CCD}}), \tag{3}$$

which prevents nodes from penetrating the ground or ceiling.

Starting from $\alpha = 1$, we iteratively shrink the step size:

$$\alpha \leftarrow \frac{1}{2}\alpha, \tag{4}$$

until the Armijo condition is satisfied or α becomes sufficiently small.

If no valid step size is found, the update is rejected.

Finally, the position is updated as:

$$\boldsymbol{x} \leftarrow \boldsymbol{x} + \alpha \boldsymbol{p}. \tag{5}$$

5 Velocity Update

After the Newton solve converges:

$$\boldsymbol{v}^{n+1} = \frac{\boldsymbol{x}^{n+1} - \boldsymbol{x}^n}{h}, \quad \boldsymbol{v}^{n+1} \leftarrow d \cdot \boldsymbol{v}^{n+1}$$

where $d \in [0, 1]$ is a velocity damping coefficient.